What Building a Network Emulation Platform Taught Me About Digital Twins

Craig Johnson
Principal Architect, Forward Networks
NANOG 98, Miami, October 2026

Safe to apply?

A routing change on one router.

The ticket: tidy the export policy on ce-svc. The services block is 10.20.30.0/24.

```
Current:   ip prefix-list SVC-OUT-ipv4 seq 100 permit 10.20.30.0/24 le 32
```

```
Proposed:  ip prefix-list SVC-OUT-ipv4 seq 100 permit 10.20.30.0/24
```

Not safe.

Without le 32, neither /25 that ce-svc advertises matches.

The BGP session stays up. The service is gone. Ask me how I know.

An emulator is not a digital twin

I built Skyforge to run disposable networks for SEs.
Building it taught me how an emulated environment differs from a digital twin.

The question determines the evidence I need.
A digital twin, as I use the word: a model built from the collected state of the real network.

Emulation execute the software	Behavioral digital twin compute from a model

Why I built it (4) → How it is built (5–7) → Lab vs twin (8–16) → Change control (17–22) → AI and close (23–25)

Why I built Skyforge

A disposable environment for SEs, without configuring it by hand every time.

Disposable

An SE needs a network to show or practice on, and then needs it gone.

Multi-user, multi-node

Many people at once, on a cluster of hosts, each with their own lab.

Not configured by hand

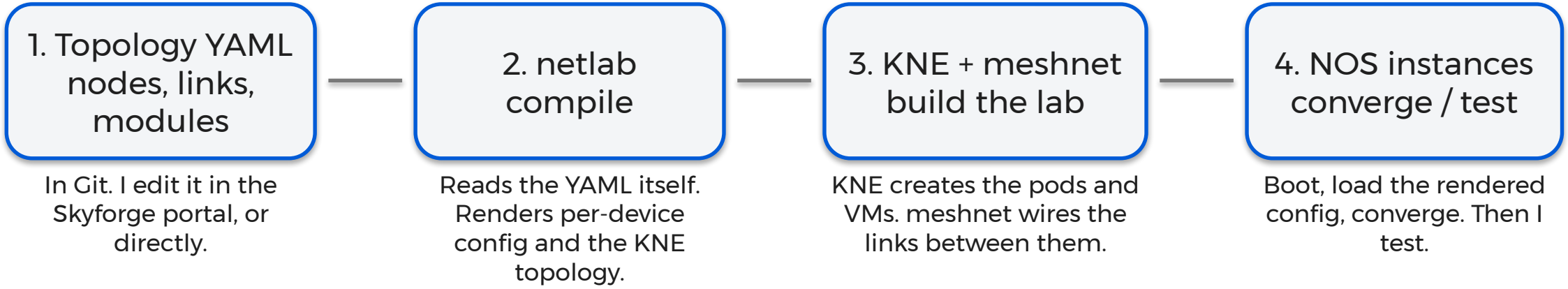
Describe the network once and get the same network every time.

I got the platform wrong twice. The fix was always the same: describe the result.

An internal platform, separate from Forward's products. When is a lab enough, and when do I need a model?

One declaration, a running network

In Skyforge, netlab reads the YAML itself; KNE and meshnet build the lab. All of this is the Skyforge portal's pipeline.



A repeatable declaration gives me a repeatable starting point.
I still have to wait for protocols and verify the result.

Forward's twin isn't in this path. It collects from the running lab afterwards.

What the declaration looks like

About 150 lines for eleven nodes. Every line is intent; none is device config.

topology.yml (excerpt)

```
defaults.device: eos
provider: kne

routing:
  prefix:
    SVC-OUT:
      - { sequence: 10, action: permit,
          ipv4: 10.20.30.0/24, max: 32 }
  policy:
    SVC-OUT:
      - { sequence: 10, action: permit,
          match: { prefix: SVC-OUT },
          set: { community: { standard: ['65020:100'] } } }

groups:
  pe:
    members: [pe1, pe2]
    module: [ospf, bgp, mpls, vrf]
    mpls.ldp: True
    mpls.vpn: True

links:
- ce-svc: { ipv4: 10.100.2.1/30, bgp.policy.out: SVC-OUT }
  pe2:    { ipv4: 10.100.2.2/30, vrf: cust_a }
- pe1-p1
```

what netlab renders for ce-svc (EOS)

```
route-map SVC-OUT-ipv4 permit 10
  match ip address prefix-list SVC-OUT-ipv4
  set community 65020:100
ip prefix-list SVC-OUT-ipv4 seq 100 permit
  10.20.30.0/24 le 32
router bgp 65020
 address-family ipv4
  neighbor 10.100.2.2 route-map SVC-OUT-ipv4 out
```

the change: delete "max: 32" in the template

```
-ip prefix-list SVC-OUT-ipv4 seq 100 permit 10.20.30.0/24 le 32
+ip prefix-list SVC-OUT-ipv4 seq 100 permit 10.20.30.0/24
```

Nobody typed the device config. netlab chose seq 100 and the -ipv4 names. A one-word edit to the template is one line of diff.

The nodes weren’t the limit. A controller was.

The lab is where cost and scale live. The twin is where the network’s real size lives.

What we measured

100 cEOS Ready in 374 s.

250 stalled at 220: the operator ran out of memory.

After a fix, 500 of 500 in 925 s, nodes at 13% CPU.

What limited us

The cEOS operator makes a 4096-bit key per device, one at a time.

The 136-node lab with the PA-VM was runtime-ready in 19 min 25 s. With four apply-phase switches on: 15 min 28 s. One run each.

Cloud bursting

Rent capacity for a big lab, run it, tear it down. We built the AWS island, used it, and destroyed it.

Tested to 500 cEOS on two nodes. Nothing larger.

What only an emulator gives me

All of it observed, in a lab I constructed.

The real software	BGP stayed Established while both /25s were withdrawn.
Real boot and transition timing	Eleven pods took about 2.5 minutes to be Ready.
Its API and CLI behaviour	Does the NOS accept this command and report the expected state?
What nobody modelled	A management default route on every router explained the timeout.

The lab can be right about the wrong network

Real NOS

Real config

Real protocols

Outside the lab:

Learned routes

Current failures

External peers

Return paths

Policy dependencies

A passing test covers the world I built.

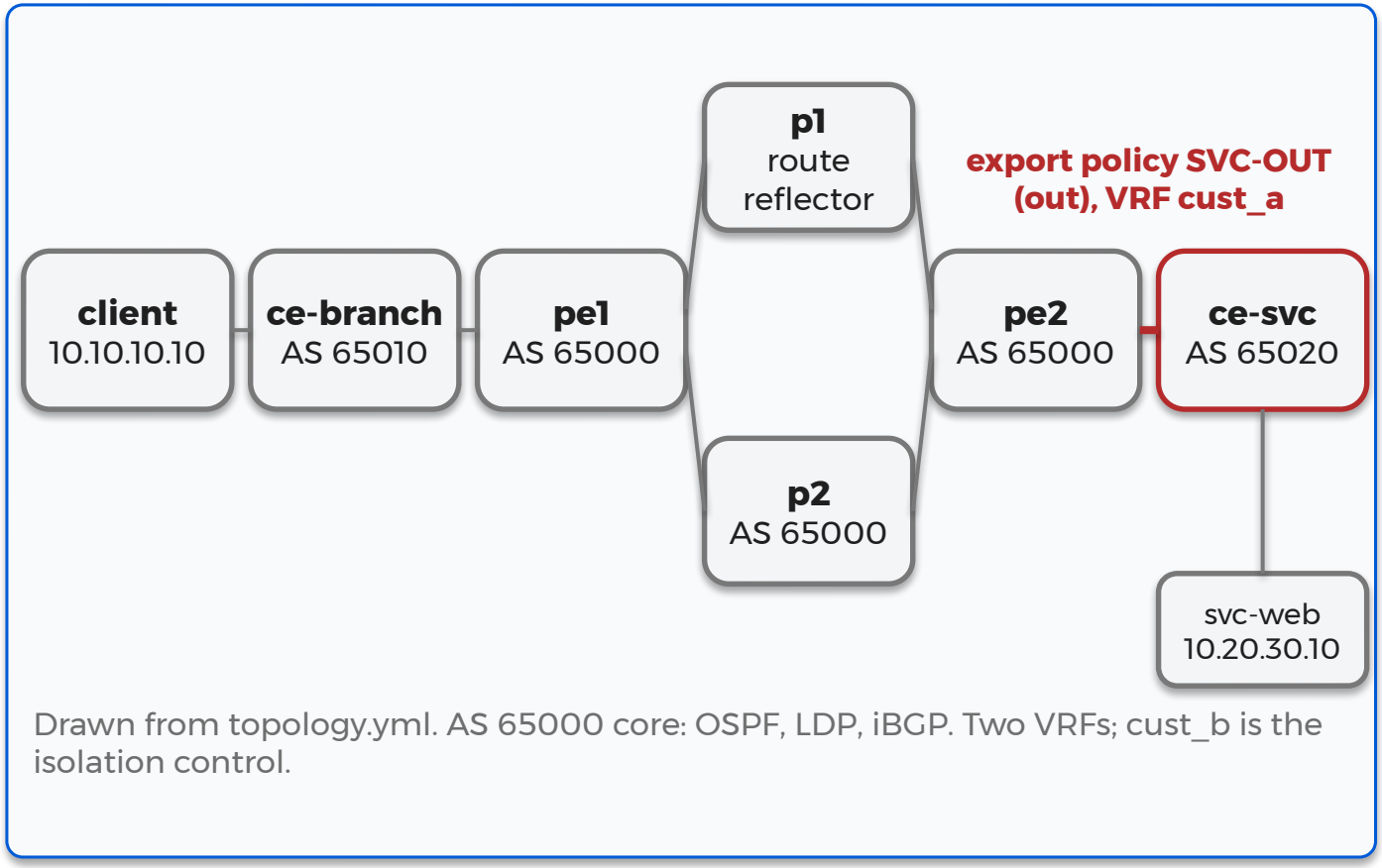
An emulator and a digital twin differ in kind

Neither is a better version of the other.

	Emulator	Digital twin
Source of truth	The topology I declare	Config and state collected from the real network
What runs	The actual NOS software	A model of supported behaviour
The question	What does this software do?	What would the real network do?
Scope	The lab I build	The whole collected network
Cost to ask	Boot pods and VMs: minutes	A snapshot and a query
Evidence	Observed: commands, timeouts	Computed: paths from a snapshot
Stays current by	Being rebuilt from the declaration	Being re-collected

Eleven nodes, one policy boundary

Provider MPLS L3VPN. The export policy is on the ce-svc to pe2 eBGP session, inside the customer VRF.



Drawn from topology.yml. AS 65000 core: OSPF, LDP, iBGP. Two VRFs; cust_b is the isolation control.

Branch client
10.10.10.10

HTTPS service
10.20.30.10:443

Services advertise two /25s:
10.20.30.0/25
10.20.30.128/25

Assumption: no covering /24, and no default route in the customer VRF.

Skyforge: what I observed

Run 2, 2026-10-04 (run 1 identical). A management default route is why the failure is a timeout, not a refusal.

```
BEFORE  baseline, run 2
ce-branch  show ip route 10.20.30.10   09:07:58Z
 B E  10.20.30.0/25 [200/0]
        via 10.100.1.2, Ethernet2
ce-svc  show ip bgp summary   09:07:46Z
Neighbor   AS     Up/Down  State  PfxRcd PfxAcc PfxAdv
10.100.2.2 65000 00:01:51  Estab  4      4      2
ce-svc  ... advertised-routes   09:07:58Z
 * >  10.20.30.0/25     10.100.2.1  65020 i
 * >  10.20.30.128/25   10.100.2.1  65020 i
```

```
AFTER  candidate, same commands, run 2
ce-branch  show ip route 10.20.30.10   09:13:19Z
 S  0.0.0.0/0  directly connected, Management0
    (no /25: only the management default)
ce-svc  show ip bgp summary   09:13:08Z
Neighbor   AS     Up/Down  State  PfxRcd PfxAcc PfxAdv
10.100.2.2 65000 00:03:03  Estab  4      4      0
ce-svc  ... advertised-routes   09:13:20Z
 (empty: nothing advertised to pe2)
session up 3 min: it was reset by the reconfigure
```

```
client  curl https://10.20.30.10/     started (UTC 2026-10-04)
baseline   09:08:15Z  http_code=200 curl_exit=0
candidate  09:13:38Z  http_code=000 curl_exit=28  (timeout)
restored   09:17:11Z  http_code=200 curl_exit=0
```

A route lookup explains the failure.

The HTTPS test checks the requirement.

Predict: the evidence I need

Forward 26.9, EOS: Predict modelled this one-line edit. Raw skill output, not a UI screenshot.

```
Forward Predict, run 2: network 5089
base snapshot 13984 -> predicted snapshot 13985
change set CHG-752, device ce-svc, 1 CLI line
compare-device-config  ce-svc configuration.txt
- 94: ip prefix-list SVC-OUT-ipv4 seq 100
       permit 10.20.30.0/24 le 32
+ 94: ip prefix-list SVC-OUT-ipv4 seq 100
       permit 10.20.30.0/24
```

```
TCP/443 10.10.10.10 -> 10.20.30.10  (default VRF)
baseline 13984  delivered:
  ce-branch > pe1 > p1 > pe2 > ce-svc
predicted 13985 dropped: ce-branch
return 10.20.30.10 -> 10.10.10.10 (src 443)
predicted 13985 delivered:
  ce-svc > pe2 > p1 > pe1 > ce-branch
```

```
verify-change, predicted snapshot 13985:  8 pairs newly isolated, 0 newly connected of 56
Not reassurance: no checks are attached, so "no check got worse" proves nothing. EOS only.
Coverage warnings: none beyond "read from a predicted snapshot". No warning is not proof.
```

Same intent, different evidence

Agreement supports this case. It does not establish equivalence for every network question.

Skyforge observed execution	Predict modeled behavior
Did this NOS withdraw the /25? Yes. Did the HTTPS probe fail? Yes.	Did the candidate remove the path? Yes. Which requirements changed? 8 of 56 subnet pairs.
Evidence: commands, timestamps, a curl timeout (exit 28).	Evidence: snapshots and the computed path, dropped at ce-branch.

A modeled path does not establish application health.
Neither establishes production timing.

I reach for emulation when software matters

CLI / API behavior
Does the NOS accept this command and report the expected state?

Failure handling
Does my automation retry safely after a partial configuration?

Protocol interaction
What does this implementation do during the transition?

Remember
Virtual convergence timing does not establish hardware performance.

I reach for a model when scope matters

Reachability
Which modeled source / destination classes lose a path?

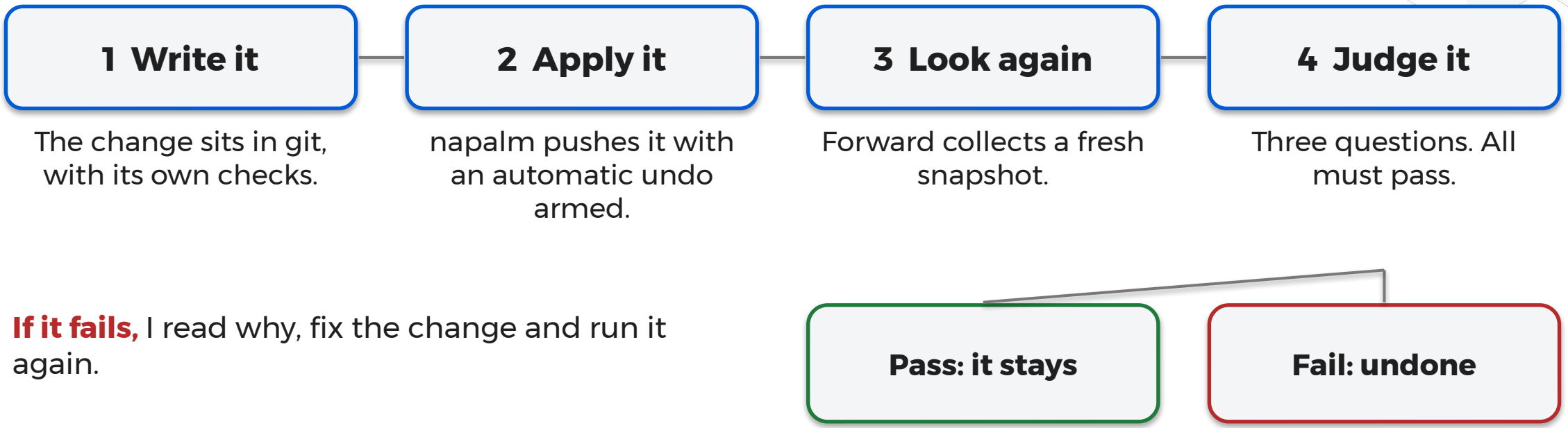
Policy
Does a routing or ACL change violate a network requirement?

Production context
What changes when I include the collected network around the lab?

Depends on
What was collected, and what Predict supports: BGP, OSPF and static routes (per org flags) and PAN-OS security rules.

A change waits until the twin has judged it

The lab proves the machinery works. The twin judges the change. I need both.



If it fails, I read why, fix the change and run it again.

Predict can check a firewall rule first. Proven live once, off by default.

Run on lab devices: cEOS, IOL, a PA-VM. Not on production.

Forward asks three things

Real numbers from the fleet40 lab, 136 devices.

1 Did anything I was running disappear?

BGP sessions, LLDP neighbours, interfaces.

Ring demo: BGP sessions 88 to 88.

2 Did anything else break?

Every check Forward already had, before and after.

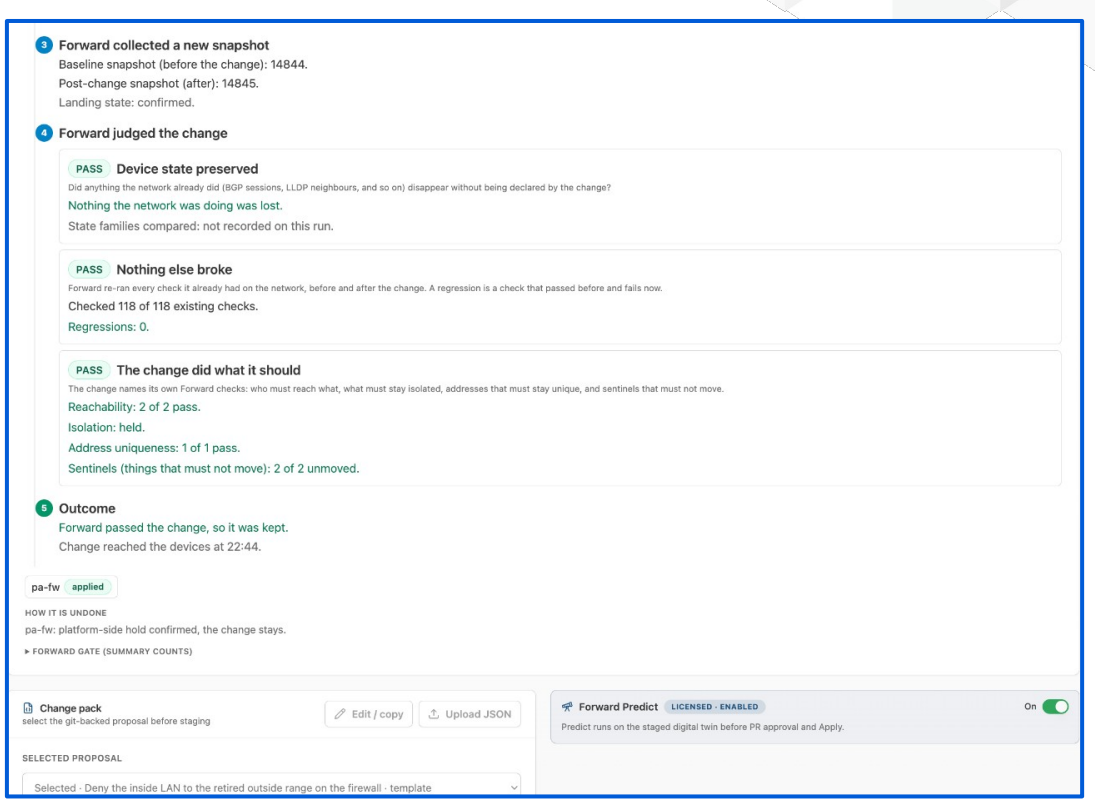
118 of 118 checks. 0 regressions.

3 Did the change do its job?

Its own checks: reach, isolation, sentinels.

Firewall fix: reach 2 of 2, sentinels 2 of 2.

A check that looks at nothing is a failure, not a pass.



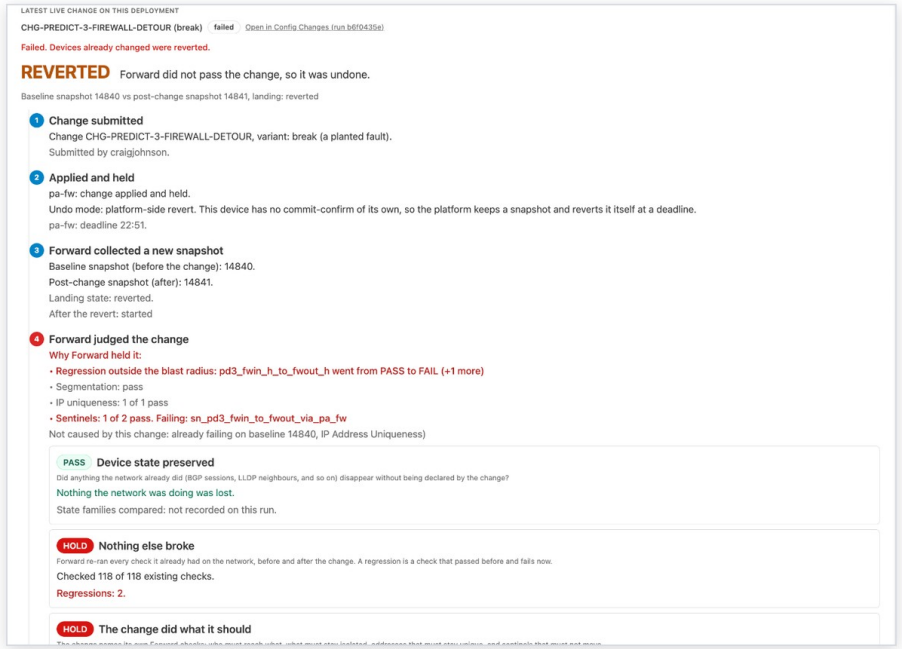
The firewall fix as Skyforge shows it (run 2aa18f9f, snapshots 14844 to 14845).

The wrong half of the range, held and undone

It reads fine. It breaks the one flow that matters.

2 Forward says no: REVERTED

The platform puts the firewall back.



Run b6f0435e · baseline 14840, after 14841

REVERTED

- Regression outside the blast radius.
- A sentinel flow no longer egresses.
- Nothing else on the network was lost.

Real screenshots from the PA-VM, 68 s, click to play. Demo 4, the ring: fault held, fix confirmed in about 2.5 minutes, 8 of 8 checks.

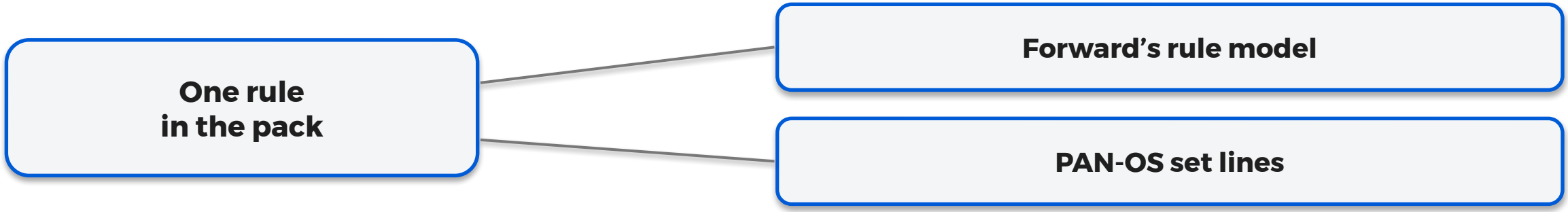


The comment on the pull request: four bullets, details folded.

A firewall can’t undo itself, so the platform does

Same flow. Different undo, and Predict reads rules, not CLI.

Device	Held by	Undone by
Arista cEOS	A timed commit	The device, on a timer.
Cisco IOL	A timed rollback	The device, on a timer.
Palo Alto	A saved config and a deadline	No commit-confirm, so a reaper restores it.



Predict reads PAN-OS rules, not CLI text. Both come from one rule, so they can’t drift.

Pre-flight, proven live (run b9a03a53): Predict held the bad rule before any device was touched. Off by default.

Say the change once: built, run on one change

Intent: what the network should do, in a form a machine can check.

Today

Device config.

Forward checks.

Predict probes.

Three shapes, one idea.

The ring demo as one intent (excerpt)

```
intent:
  links:
  - ring-b: null
    ring-c: {shutdown: true}
  expect:
    reachable:
    - {from: app, to: ring-svc}
    path:
    - {from: app, to: ring-svc,
       via: ring-d, assert: present}
    unchanged: [bgp-sessions]
    removed:
    - {adjacency: [ring-b, ring-c]}
  variants:
    break:
      links:
      - ring-c: {shutdown: true}
        ring-d: null
```

Written once

Checks, probes and device config come from it.

A fault is a patch

Two lines over the base.

Raw CLI

Marked, and Predict can’t model it.

Built, run once. One firewall change ran live from an intent. The shared pack still uses the old form.

The twin judged it right. The machinery broke.

The first real firewall run found eight bugs. Our test fake could not raise any of them.

- **Names.** PAN-OS refuses a saved-config name over 32 characters.
- **Save.** “Save config” saves the candidate, not the running config. A leftover became every later rollback point.
- **Library.** It raised an error on a commit that had worked.
- **Rule.** “Application any” with “application-default” is rejected.

The twin judged the change. The machinery had to be proven on the device.

AI helps. A person still approves.

Each piece says how far it has got.

Working today

AIOps drafts fixes and a person approves. Suggest a fix is compile-checked first.

After a hold

Forward's AI summary says why it failed. Predict can hold a firewall rule first: proven live, off by default.

Run live, once

A model drafted the rework on the demo firewall. A person approved it. Forward confirmed it. Once, not routine.

AI drafts. Predict checks. A person approves. Forward judges.

Forward and Predict never push to devices.

What I took away

The lab tells me what this software does.

The twin tells me what happens in my network.

I don’t swap them.

The lab proves the machinery. Checks gate it. The twin judges.

A check that looks at nothing is a failure, not a pass.

The question I ask now

"What am I trying to learn?"

"What evidence would change my decision?"

Patches and notes:
github.com/forwardnetworks/skyforge



Questions

Where has your lab given you
a misleading answer?

Craig Johnson

Appendix: for questions only

Not presented. Slides behind this one are for Q&A.

- All eight defects from the first firewall run.
- One more intent example: the firewall change and its planted fault.
- The comment on the pull request, both runs.
- The earlier change demo (fleet40, August).
- The approval gate for AI actions.
- Why we patch upstream, and the repo.
- Why not just containerlab.

Appendix: all eight defects from the first firewall run

Demo 3, first live PA-VM runs, 2026-10-10. Fixed in the applier and the Go renderer.

1. Saved-config name must be 32 characters or fewer. Names are now skyforge-<run8>-<hash6>.xml.
2. The PAN-OS library raises on an error response, including "no such saved file". That means absent, not failure.
3. "Save config" saves the candidate, not the running config. The applier reverts the candidate before saving.
4. The CLI library raised "commit failed" on a commit that reached 100%. After a commit error the device decides: running equals snapshot means it failed.
5. "Application any" with "service application-default" is rejected at commit. The default is now "service any".
6. Diff-only runs carry no snapshot name. The snapshot is optional for a diff.
7. An ungated apply was confirmed, but the hold stayed armed and blocked every later change to the firewall.
8. A netlab image pin that rides a config map takes effect only after the Helm upgrade completes.

Appendix: the firewall change as one intent

Built, run once on this change. Demo 3: one deny rule.

Design excerpt. The live run used the rule and the break variant only.

```
intent:
  nodes:
    pa-fw:
      firewall.policy.rules:
      - name: DENY-INSIDE-TO-RETIRED-RANGE
        action: deny
        from_zone: inside
        to_zone: outside
        src: [10.10.101.0/24]
        dst: [10.10.102.128/25]
        position: top
  expect:
    reachable:
    - {from: fwin-h, to: fwout-h}
    path:
    - {name: sn_pd3_fwin_to_fwout_via_pa_fw,
       via: pa-fw, assert: present}
  variants:
    break:
      nodes:
        pa-fw:
          firewall.policy.rules:
          - {name: DENY-INSIDE-TO-RETIRED-RANGE,
             dst: [10.10.102.0/25]}
```

The planted fault is one line.

The break variant only changes dst: 10.10.102.128/25 to 10.10.102.0/25. Everything else comes from the base.

Both consumers come from it.

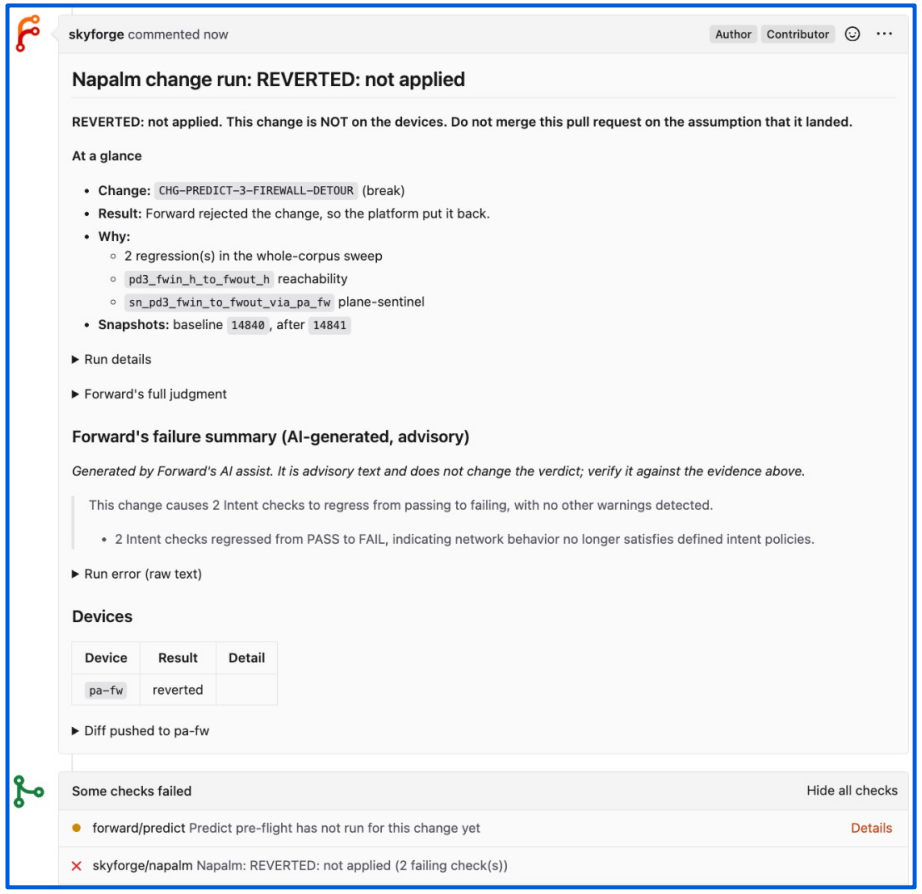
The same rule feeds Forward's rule model and the PAN-OS set lines. In the live run the checks and probes stayed in the pack, beside the intent.

Run once, not routine.

The firewall.policy plugin is ours, not stock netlab. "position" and "absent" are invented names.

Appendix: the comment on the pull request

Pull request #27 carries both. A few bullets each; the details are folded.



skyforge commented now — Author — Contributor

Napalm change run: REVERTED: not applied

REVERTED: not applied. This change is NOT on the devices. Do not merge this pull request on the assumption that it landed.

At a glance

- **Change:** `CHG-PREDICT-3-FIREWALL-DETOUR` (break)
- **Result:** Forward rejected the change, so the platform put it back.
- **Why:**
  - 2 regression(s) in the whole-corpus sweep
  - `pd3_fwin_h_to_fwout_h` reachability
  - `sn_pd3_fwin_to_fwout_via_pa_fw` plane-sentinel
- **Snapshots:** baseline `14840`, after `14841`

▶ Run details

▶ Forward's full judgment

Forward's failure summary (AI-generated, advisory)

Generated by Forward's AI assist. It is advisory text and does not change the verdict; verify it against the evidence above.

> This change causes 2 Intent checks to regress from passing to failing, with no other warnings detected.
>
> - 2 Intent checks regressed from PASS to FAIL, indicating network behavior no longer satisfies defined intent policies.

▶ Run error (raw text)

Devices

Device	Result	Detail
`pa-fw`	reverted	

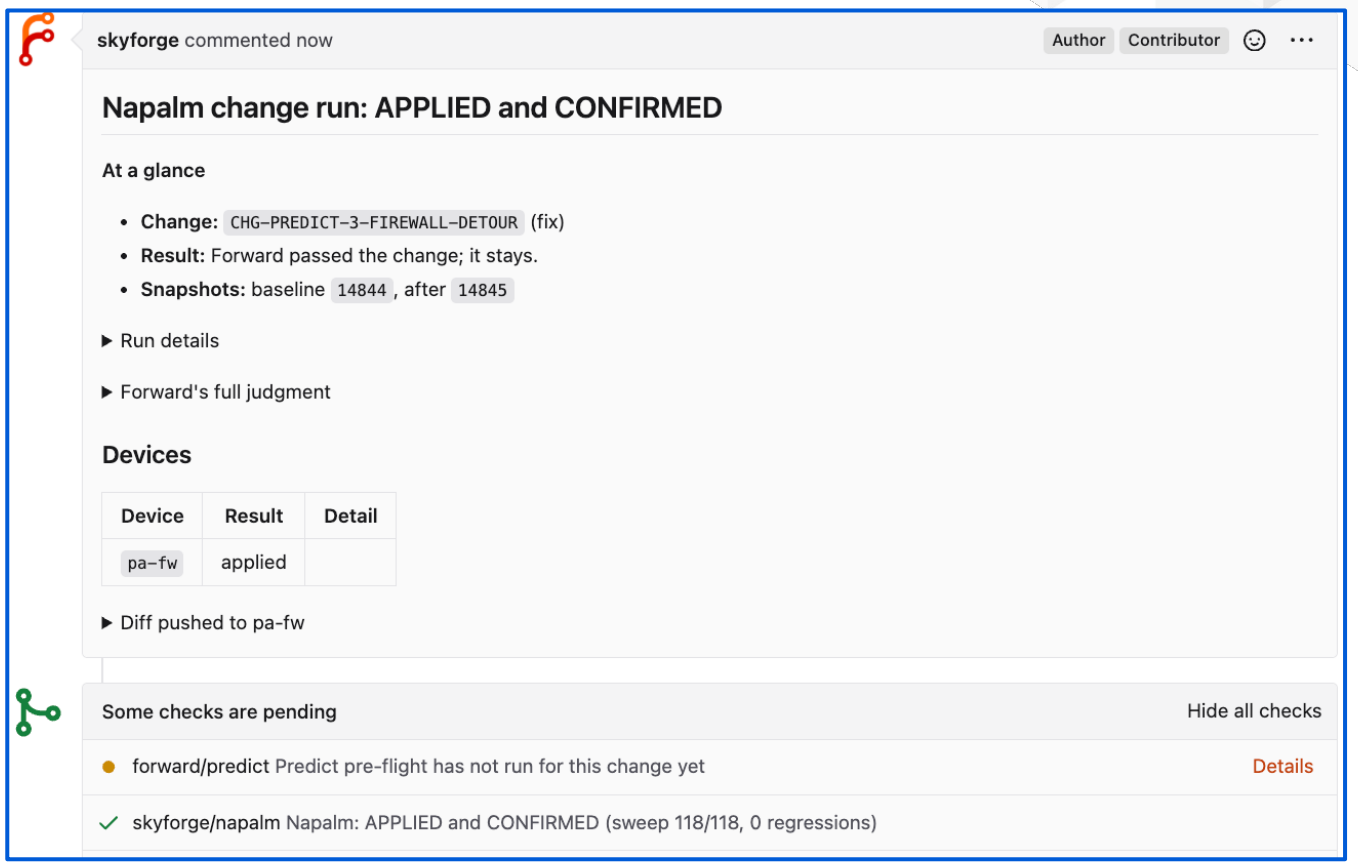
▶ Diff pushed to pa-fw

Some checks failed — Hide all checks

● forward/predict Predict pre-flight has not run for this change yet — Details

✕ skyforge/napalm Napalm: REVERTED: not applied (2 failing check(s))

Break, run b6f0435e: REVERTED, not applied.



skyforge commented now — Author — Contributor

Napalm change run: APPLIED and CONFIRMED

At a glance

- **Change:** `CHG-PREDICT-3-FIREWALL-DETOUR` (fix)
- **Result:** Forward passed the change; it stays.
- **Snapshots:** baseline `14844`, after `14845`

▶ Run details

▶ Forward's full judgment

Devices

Device	Result	Detail
`pa-fw`	applied	

▶ Diff pushed to pa-fw

Some checks are pending — Hide all checks

● forward/predict Predict pre-flight has not run for this change yet — Details

✓ skyforge/napalm Napalm: APPLIED and CONFIRMED (sweep 118/118, 0 regressions)

Fix, run 2aa18f9f: APPLIED and CONFIRMED. First live capture.

We patch upstream, in small series, on pinned versions

netlab, KNE, meshnet-cni and koko. Pristine upstream plus patch files.

netlab	**49 patches** coming	Vendor-neutral modules we need, such as firewall.policy. KNE-provider fixes. Device bring-up fixes.
KNE	**80 patches** coming	Lab services without NodePorts. VM-lane ordering. Legal Linux interface names for pod links.
meshnet-cni	**24 patches** published	Wiring links across nodes with VXLAN.
koko	**15 patches** published	Cross-node VXLAN binding fixes.

A failed apply means upstream moved, and we fix the patch. We intend to send them upstream; none is accepted yet.
Published now: koko 15, meshnet-cni 24, Arista cEOS operator 19, and three small series (catalystwan 5, catalystwan-sdk 1, nokia-grpc 1). Coming after review: netlab and KNE.

Appendix: the August change demo, fleet40

The fleet40 change demo, measured 2026-08-16 and 2026-08-17.

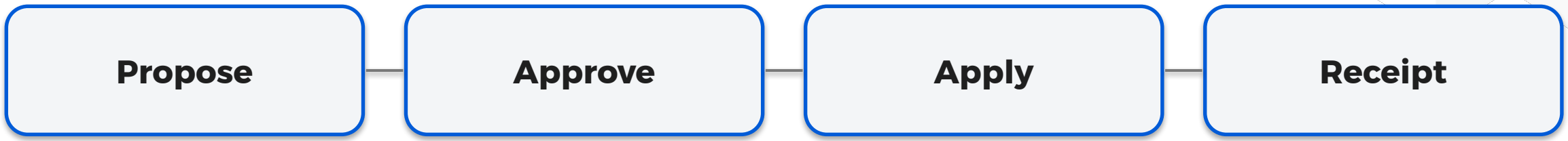


- **The intended change:** Predict said PROCEED, 82 requirements pass and 0 fail. A second person approved it. After the push, the live gate read the devices: 82 pass, 0 fail.
- **A planted fault:** Predict said HOLD, 2 pass and 80 fail. I applied it anyway, as an operator overriding the warning. The live gate said the same thing from the devices: 2 pass, 80 fail.
- **Predict never pushes.** Reset restored the baseline, and the cycle ran twice back to back.

Source: docs/fleet40-change-demo-runbook.md (measured runs; predicted snapshots 4912 and 5063).

Appendix: where the AI is allowed to act

Propose, approve at an exact head, apply, keep a receipt.



An assistant or a person drafts the change.

A person merges it in Forgejo, bound to an exact head.

Only what was approved runs. A new edit is a new approval.

Approval issues one single-use, expiring receipt. Any drift voids it.

Reversible chat actions can run for the person asking. Irreversible actions are refused without a reviewed approval.

Today the gate is armed for one action (extend a lease), and each action is enabled on its own. Arming one never arms another.

Appendix: why not just containerlab?

It is a very good tool. It answers a different question than the one I had.

containerlab	What I needed
• One engineer on one host • You run, debug and tear it down yourself • Excellent for building and testing a topology	• Many SEs sharing one cluster • Labs that span several hosts • Leases, sharing and lifecycle through a portal • Containers and VM-based NOS side by side

I kept what fit: netlab for topology and config, KNE for pods and VMs.

The next slide is what happened when I tried to bend a one-host tool into a multi-user platform.